Lecture => 8

Objects → Key value pairs

```
const obj1 = {
    name : "BKK",
    age : 25,
    email_id : "BKK@gmail.com"
};
```

→ Internally it is stored like this

```
"name" : "BKK",
"age" : 25;
```

// Object destructuring

```
const {name, age} = obj2;
console.log(name, age);
```

// Note => We can not use for of loop directly on objects

note `Object.keys(obj2);` → will return array of keys

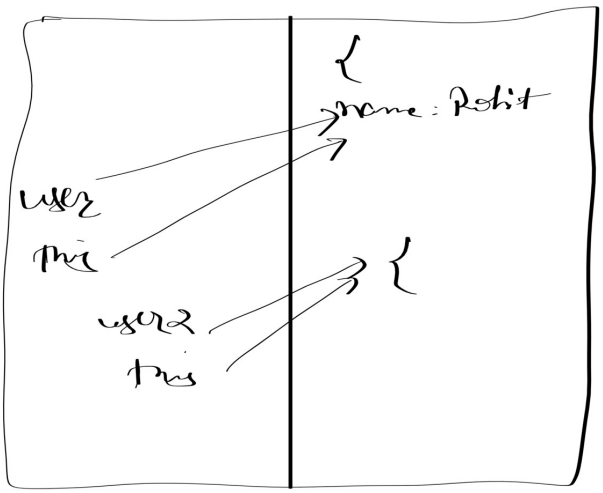
Then on this we can apply for of loop.

```
for (let keys in obj2) {
    console.log(keys, obj2[keys]);
}
```

Destructing of obj

```
const { property1, property2 } = obj3;
console.log(property1, property2);
```

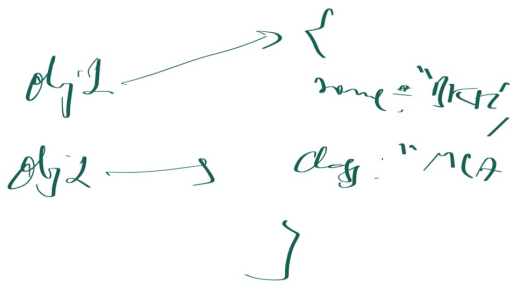
⇒ uses of this keyword



as we know that object is pass by reference

```
const Obj1 = {
    name: "Rohit",
    class: "MCA"
}

const Obj2 = obj1;
```



But if we want seperate obj then

well

```
const obj2 = {...user};
```

But this will help to do for 1 level

for 2 level it will not work

↳ obj{

obj{

}

}

This is also known as shallow copy

To resolve this we use

structuredClone() method

```
const obj2 = structuredClone(obj);
```

How array is stored internally

```
const arr = [1, 2, 3, 4, 5]
```

→ {

0 : 1

1 : 2

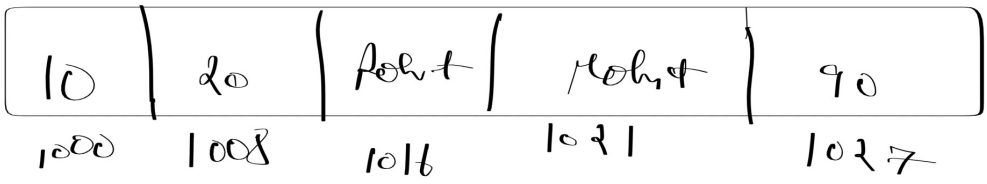
2 : 3

3 : 4

4 : 5

}

But how it is stored internally lets see

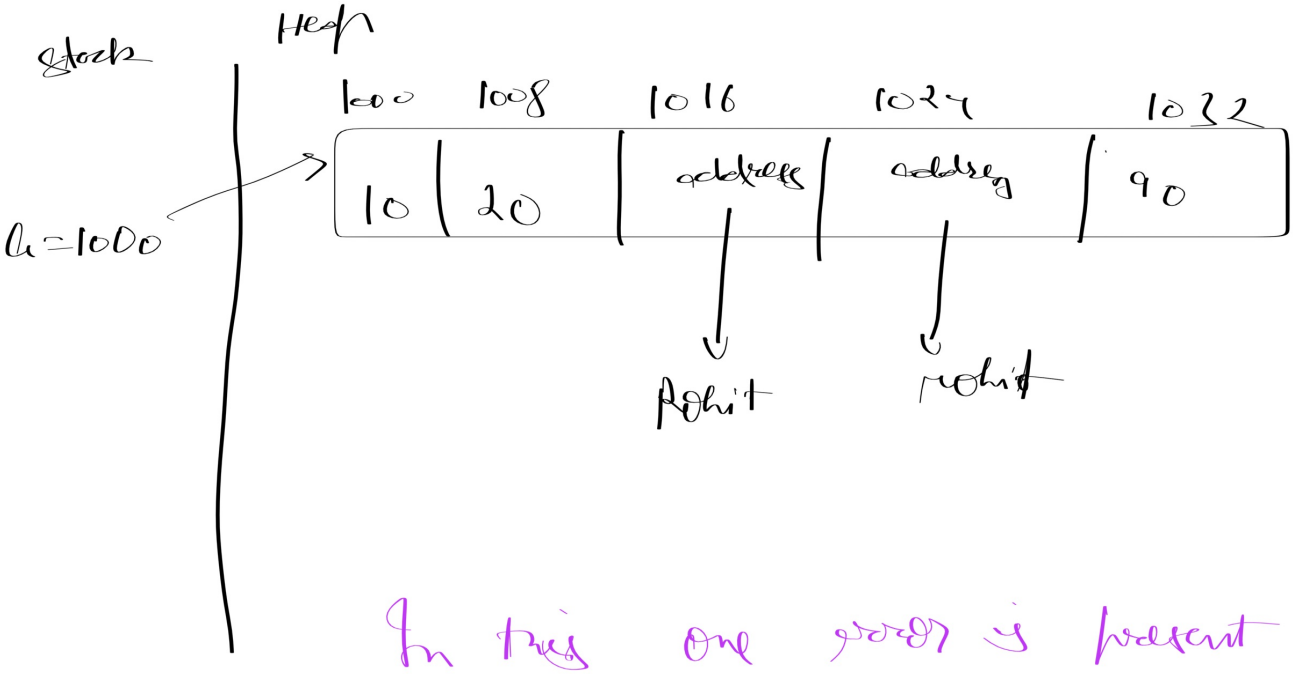


Address & bytes

Base address + index * byte of data

Not possible in this

Then what will be done



In this one more is present

Suppose heap memory for that hostelor is full

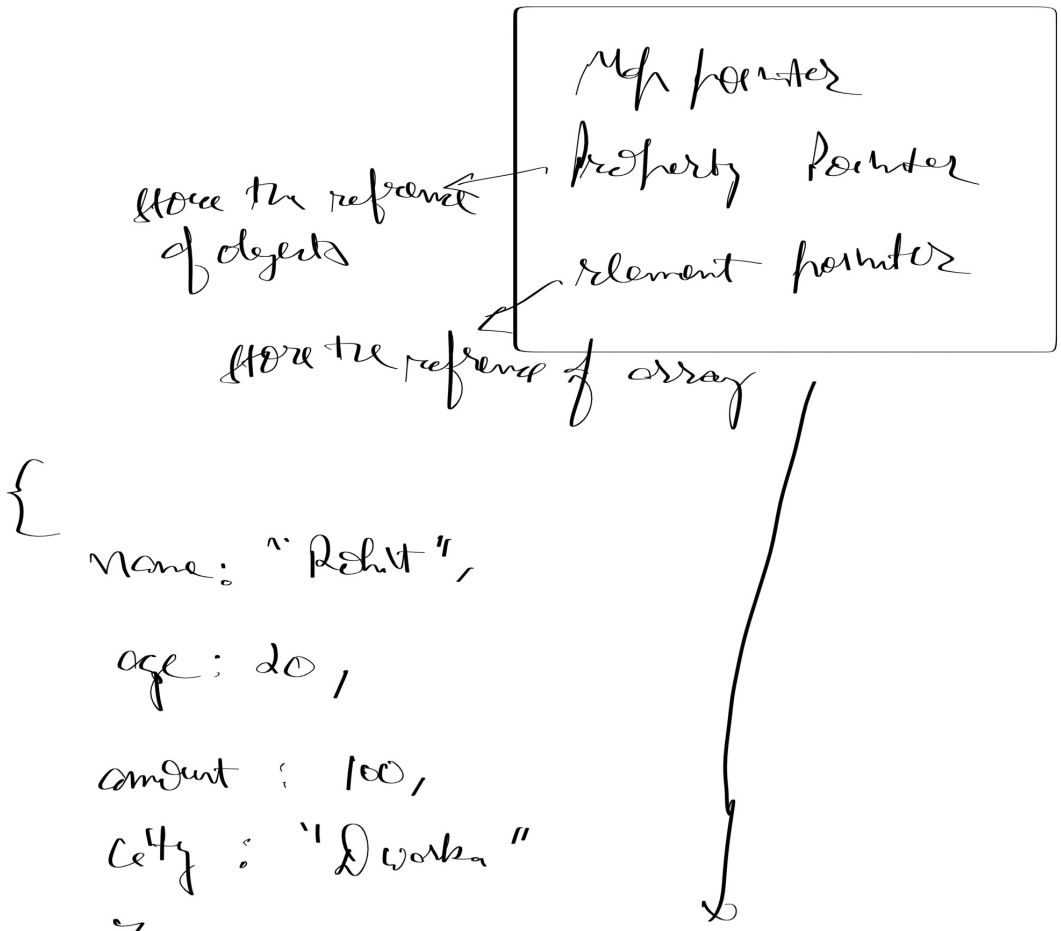
then heap will see where to store/copy the whole array elements and store it there

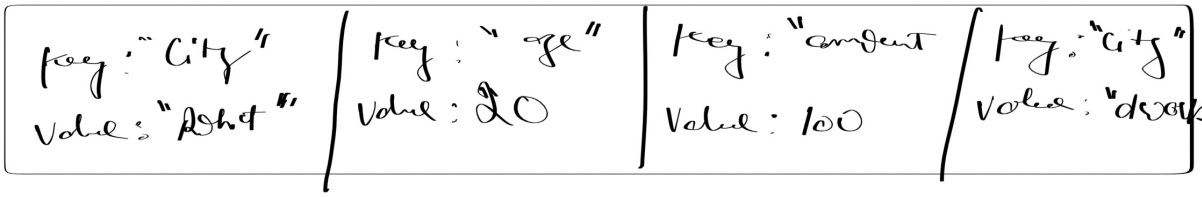
& last in that reference

address will change when is not possible
at it use const ver ?

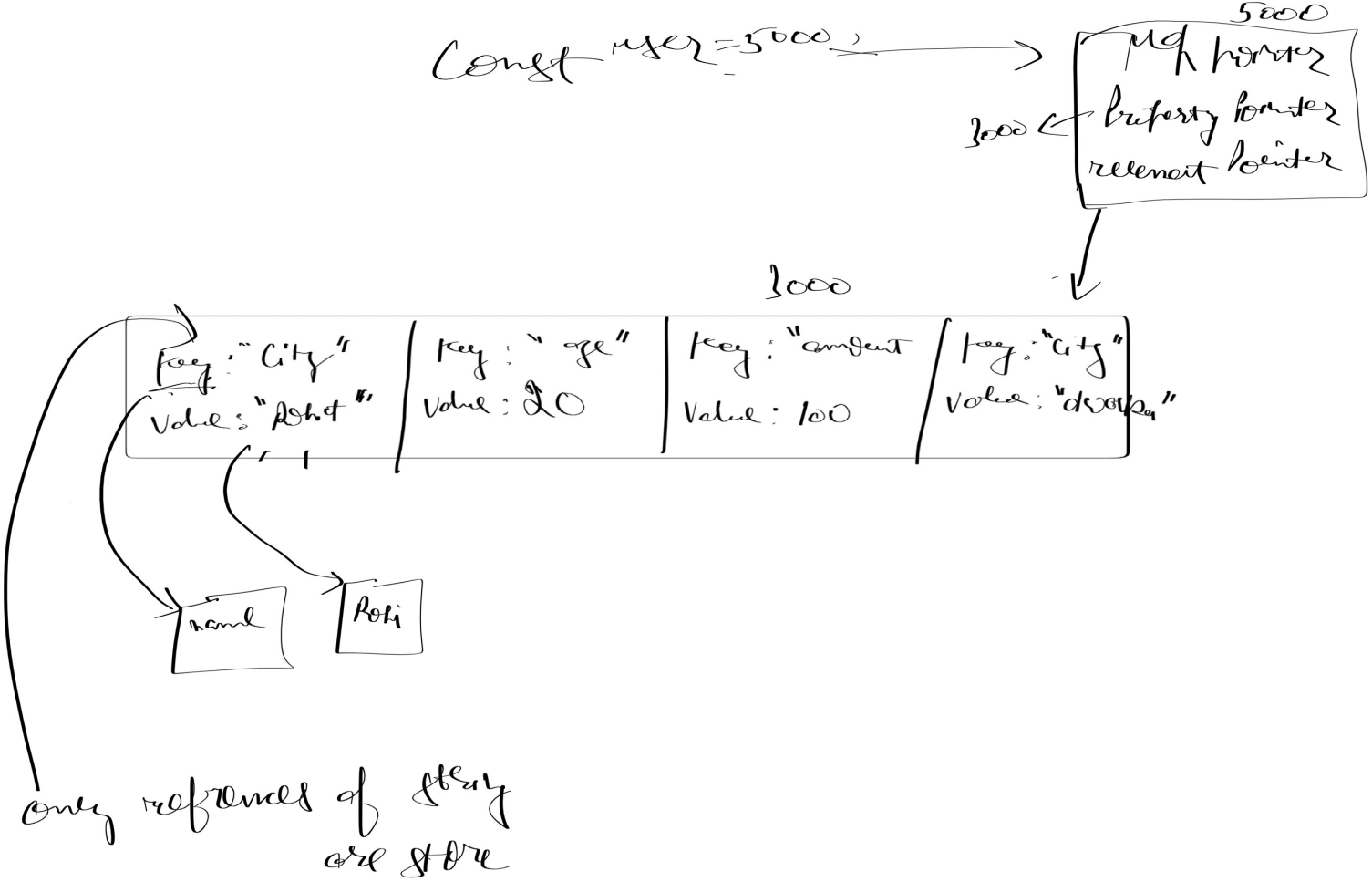
How

First see how objects are stored





Best when changes happen for feature
then it will take more memory
& there is not much available.


Const user = 5000;
5000
MQ header
property pointer
relevant pointer
3000
3000
key: "City"
value: "Rohit"
key: "age"
value: 20
key: "amount"
value: 100
key: "City"
value: "dsxaq"
name
Rohi
only references of keys are store